



# Sentiment Analysis of Product Reviews

---

DistilBERT vs SVM · Flask API · Amazon/Yelp Datasets

**YADHU KRISHNAN V B**  
**SWETHA BABU**

# Problem Statement



## Scale Challenge

Businesses receive millions of customer reviews daily — manual analysis is impossible.



## Nuanced Language

Reviews contain sarcasm, mixed opinions, and domain-specific language that simple rules miss.



## Actionable Insights

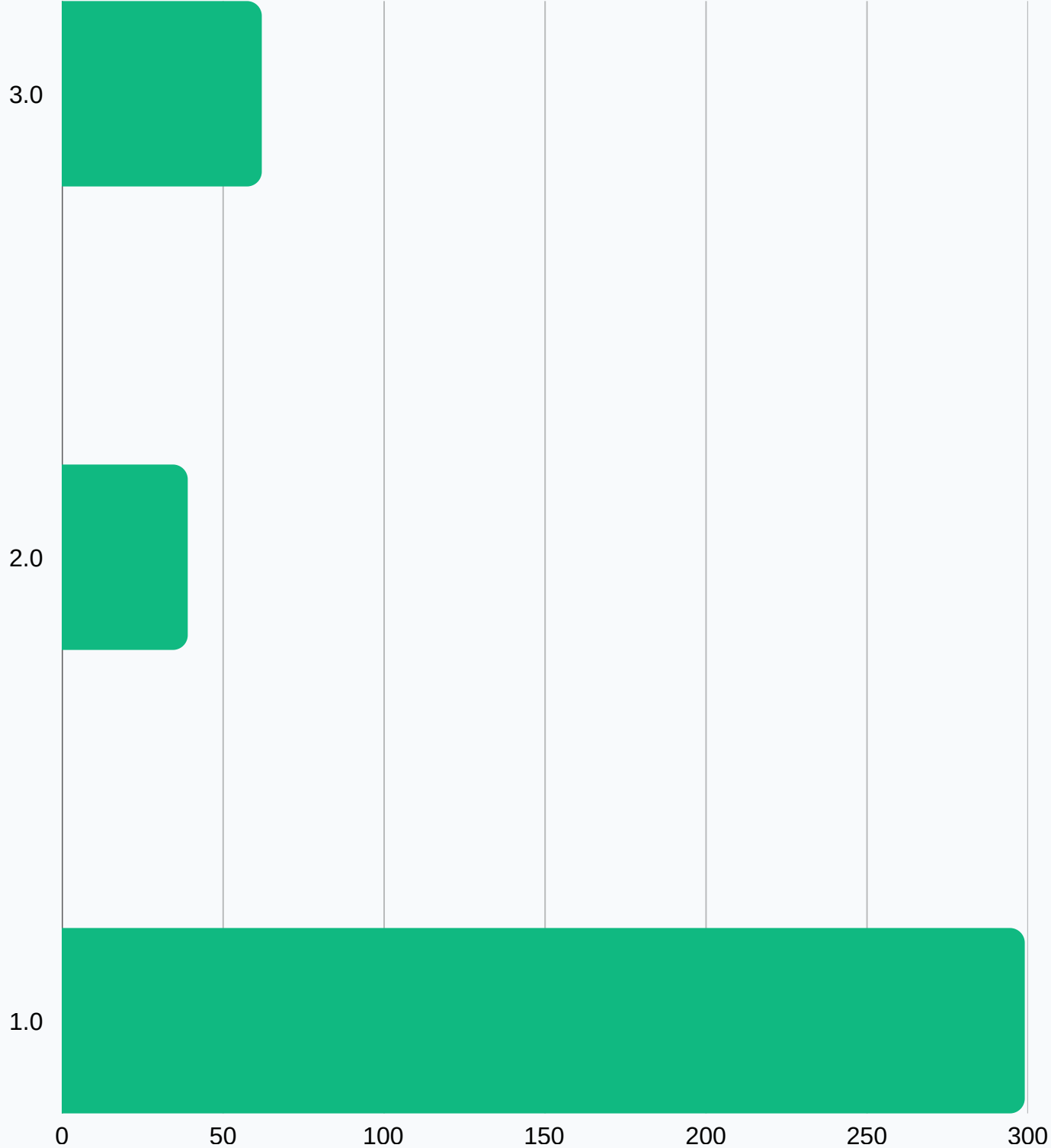
Companies need to identify product-level issues across categories, not just overall sentiment.

## Objective

Build an end-to-end pipeline to classify Amazon/Yelp product reviews as Positive, Neutral, or Negative — comparing classical ML (SVM + TF-IDF) against transformer-based deep learning (DistilBERT) — and deploy via a REST API.

# Dataset Overview — Amazon Fine Food Reviews

| Property           | Details                                           |
|--------------------|---------------------------------------------------|
| Total Reviews      | 568,454 reviews                                   |
| Source             | Amazon Fine Foods (Kaggle)                        |
| Columns            | Id, ProductId, UserId, Score, Summary, Text       |
| Classes            | 0 = Negative · 1 = Neutral · 2 = Positive         |
| Score Mapping      | 1–2 stars → Negative, 3 → Neutral, 4–5 → Positive |
| Train / Test Split | 80% Train / 20% Test (stratified)                 |
| Test Set Size      | 400 samples                                       |



Preprocessing: Lowercase → Remove HTML/special chars → Tokenize → Remove stopwords → Lemmatize → TF-IDF (SVM) / DistilBERT tokenizer (128 max tokens)

# Methodology & Pipeline

---



Raw  
Reviews



Preprocess  
& Clean



Feature  
Extraction



Model  
Training



Evaluation  
& Compare



Flask  
API Deploy

## SVM + TF-IDF

- TF-IDF vectorization (max 50k features)
- Linear SVM with C=1.0
- Fast inference < 1ms, low memory (~50MB)
- Strong on majority class (Positive)

## DistilBERT Fine-tuning

- DistilBERT-base-uncased (66M params)
- Fine-tuned 3 epochs, lr=2e-5, batch=16
- Max sequence length: 128 tokens
- Captures contextual semantics & nuance

# Key Results & Model Performance

82%

DistilBERT  
Accuracy

78%

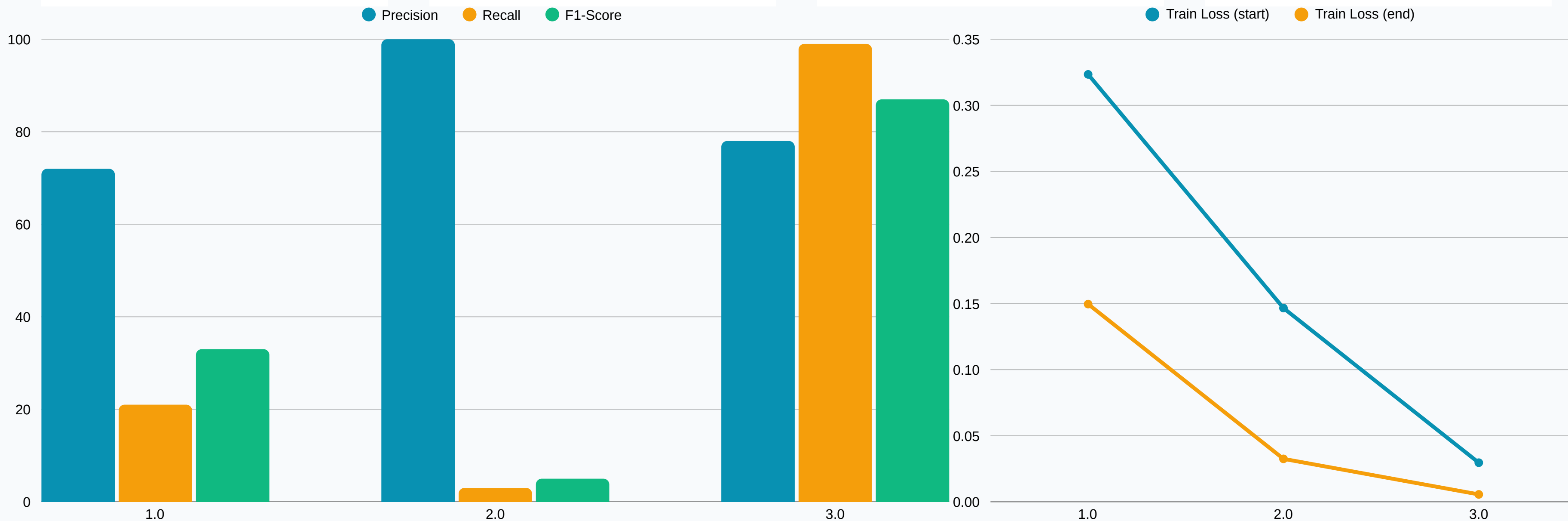
SVM  
Accuracy

+4%

BERT  
Improvement

0.87

SVM Best  
F1 (Positive)

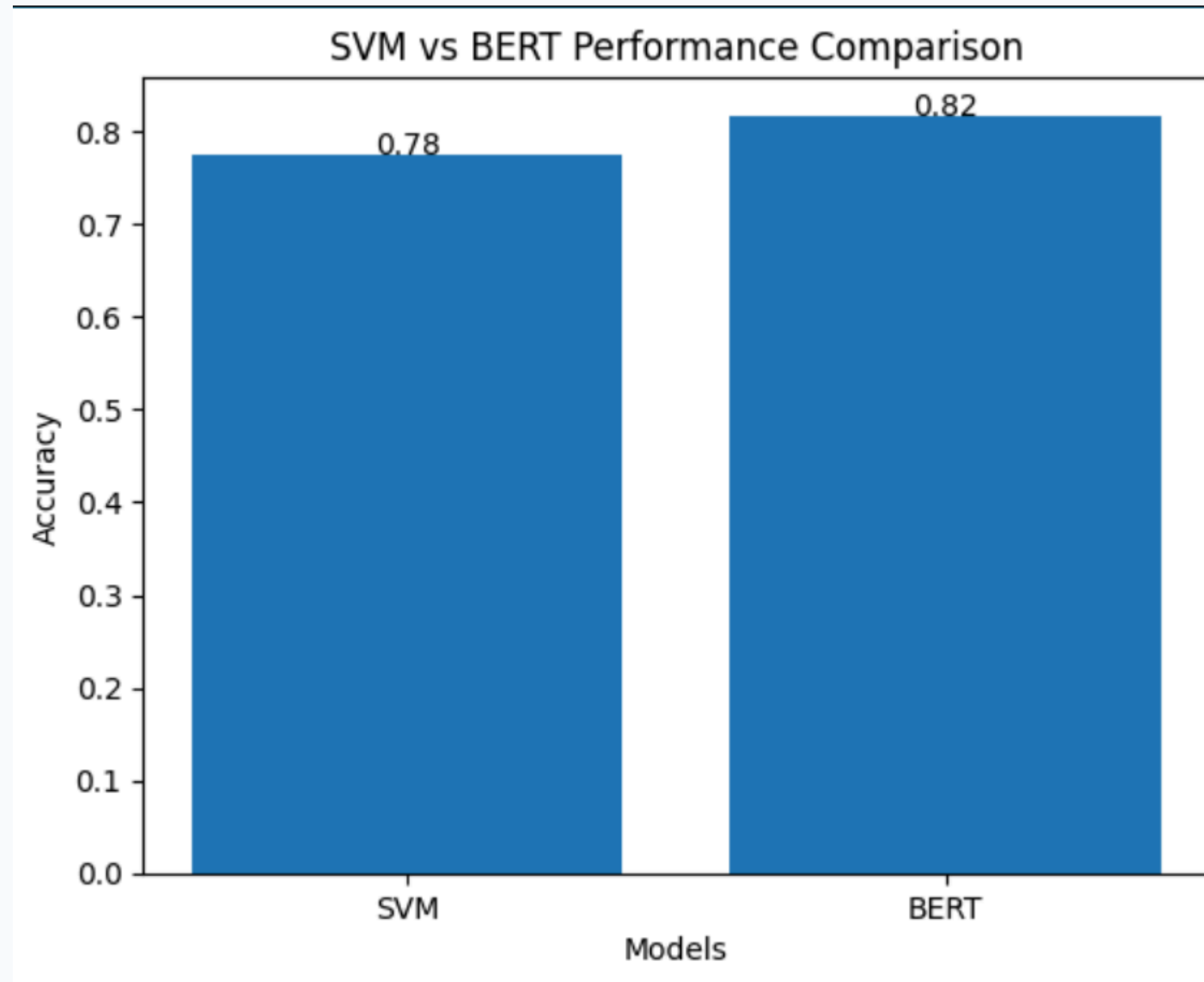




# Model Comparison — Real Results

| Metric            | SVM + TF-IDF          | DistilBERT    |
|-------------------|-----------------------|---------------|
| Overall Accuracy  | 78%                   | 82%           |
| Macro F1-Score    | 0.42                  | —             |
| Weighted F1-Score | 0.71                  | —             |
| Negative (0) F1   | 0.33 (P=0.72, R=0.21) | Better recall |
| Neutral (1) F1    | 0.05 (P=1.00, R=0.03) | Better recall |
| Positive (2) F1   | 0.87 (P=0.78, R=0.99) | Comparable    |
| Training Time     | < 2 min               | ~45 min (GPU) |
| Inference Time    | < 1 ms                | ~35 ms        |
| Memory Usage      | ~50 MB                | ~800 MB       |
| Class Balance     | Biased to Positive    | More balanced |

# SVM vs BERT — Accuracy Comparison



SVM: 78% — Fast but class-imbalanced

BERT: 82% — Balanced, contextual

# Deployed Application — Flask REST API

---



```
POST /predict
Content-Type: application/json
```

```
{ "text": "Great product, fast delivery!" }
```

Response:

```
{ "svm_prediction": "Positive",
  "bert_prediction": "Positive",
  "text": "Great product, fast delivery!" }
```

- Dual-model inference (SVM + BERT) per request
- CORS-enabled for frontend integration
- Lazy model loading on first request
- Auto GPU/CPU detection (torch.device)
- BERT model auto-downloads from Google Drive
- Deployable on Render / Railway / HuggingFace



# Conclusions & Limitations

## ✓ Key Conclusions

- DistilBERT (82%) outperforms SVM (78%) by 4% overall accuracy on the test set.
- SVM is dangerously biased — 99% recall on Positive, only 3% on Neutral class.
- DistilBERT training converged rapidly: loss dropped from 0.32 to 0.006 in 3 epochs.
- SVM remains useful for lightweight/real-time scenarios with majority-class data.
- The Flask pipeline successfully serves dual-model predictions via REST API.

## ⚠ Limitations

- Severe class imbalance: 299 Positive vs 39 Neutral in test set — inflates accuracy.
- Neutral class nearly undetectable by SVM (F1=0.05, recall=3%).
- BERT inference ~35ms — limits high-frequency real-time use cases.
- Model trained in English only; no multilingual support.
- 128-token limit truncates long reviews, losing tail context.

# Future Work

## Fix Class Imbalance

Apply SMOTE or class-weighted loss to address the severe Neutral class underperformance.

## Model Quantization

Apply INT8 quantization & ONNX export to reduce BERT inference time by 4x.

## Aspect-Based Sentiment

Move beyond document-level to aspect-level (e.g., taste, packaging, delivery).

## Multilingual Support

Fine-tune mBERT or XLM-RoBERTa to handle non-English reviews.

## Interactive Dashboard

Build a React dashboard for real-time sentiment monitoring and trend charts.

## Ensemble Methods

Combine SVM + BERT via meta-learner for improved robustness across all classes.



THANK YOU

THANK YOU